Joshua Peek 21081733
CST310
Project 3 & 4 (cont at bottom)
7/6/2025

Book 16.xlsx (link because file is subject to change)

1. List the objects in the foreground, background, and in-between.

   Excel Sheet Provided

2. Identify the main objects in the scene (e.g., buildings, trees, bridge).

   Excel Sheet Provided

3. Describe key characteristics of the scene (e.g., ambience).

   Excel Sheet Provided (notes)

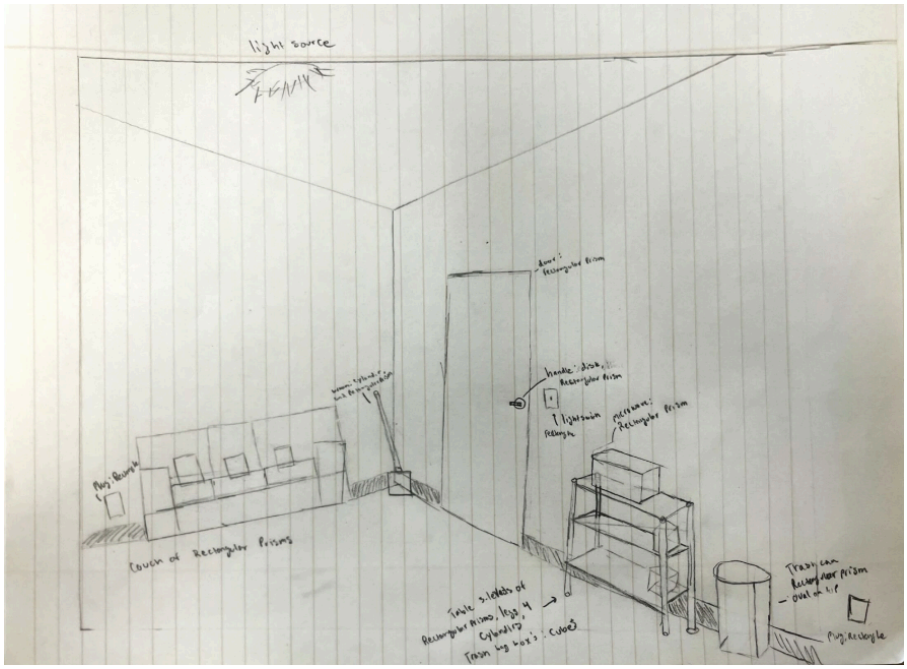
4. Explain how you would approach rendering each object in the scene. For example, an elementary school would be represented by a set of cubes with trapezoidal rooftops. A tree would be represented by a cylinder with a reverse pyramid on top.

   Excel Sheet Provided (each object is described by a corresponding shape or collection of shapes that aptly define the object)

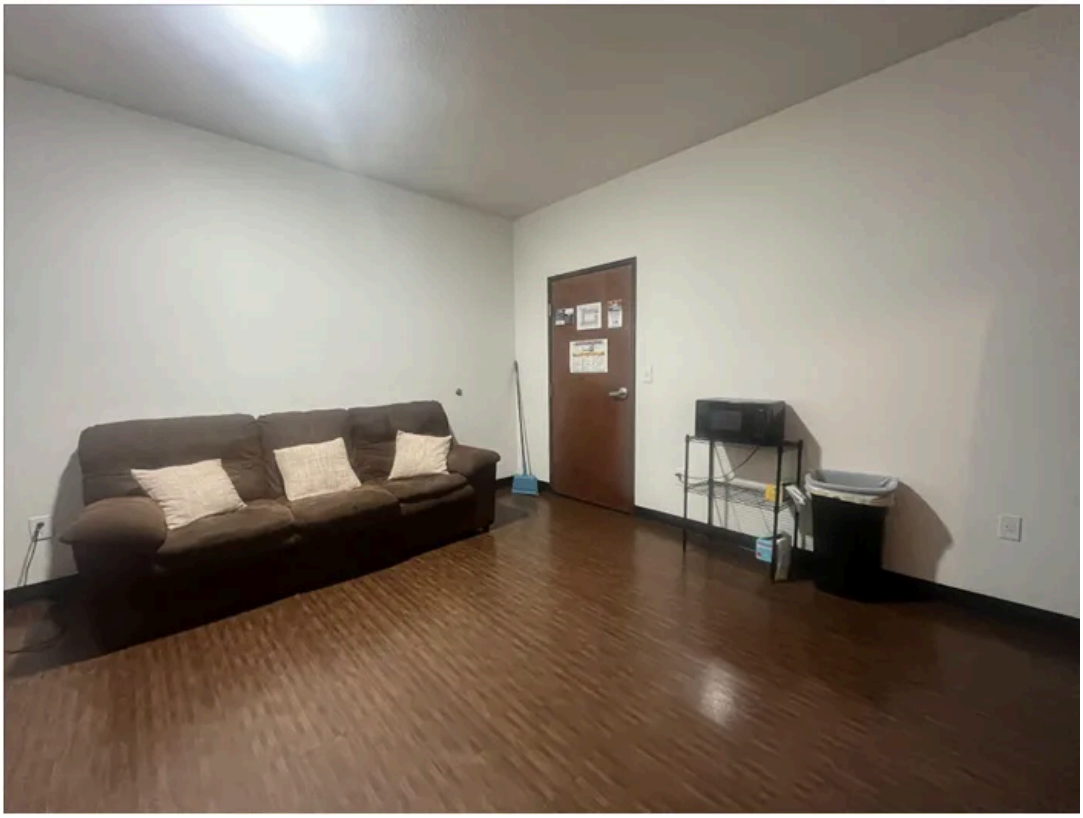
5. Rank all the objects in the scene in order of rendering difficulty. **NOTE:** You will have to render at least 10 different objects in the next project.

   The Excel Sheet provided is in order of difficulty, with the exception of rows 14 and 15 (table and trash can), which should be #1 and #2 due to the difficulty of ‘combining’ different geometric shapes to form a complete object.

6. Draw a geometrical representation of the picture on paper, by hand, using a pencil only.

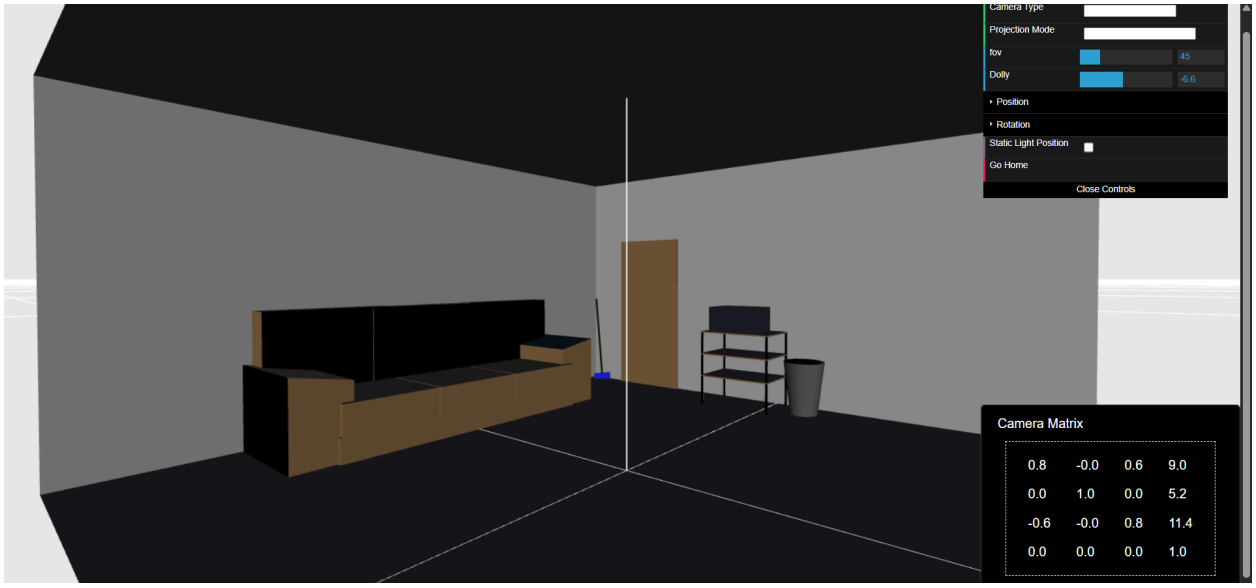


Show the original and the hand-drawn images side by side.



Project 4

1. Use the hand-drawn scene created in Project 3 and reproduce each object using primitives.
2. Use the virtual camera to display the desired perspective of the scene.
3. Use 3D primitives and various transformations like translation, rotation, and scaling, to reproduce the main objects of the scene. Maintain the relative size of all objects. Use solid colors for now (e.g., green for tree leaves, blue for sky).



Below is a detailed analysis of each object in the scene based on the provided index.html file, reflecting the state as of 08:56 PM MST on Sunday, July 06, 2025. I'll (Grok) cover the mathematical characteristics, primitives used, transformations, shaders, and camera details as requested.

Objects in the Scene

1. Floor

- **Mathematical Characteristics**:
  - Represented as a plane with dimensions 2000x100 units (from Floor.js, assuming a rectangular base).
  - Likely a 2D grid or single quad scaled to these dimensions.
  - Vertices are typically defined in the x-z plane with y = 0.
- **Primitives Used to Render**:
  - Triangles (likely two triangles forming a quad, or a triangle strip/mesh for larger floors).
- **Transformations Used and Explanations**:

    - floorTransform = mat4.create(); mat4.identity(floorTransform); mat4.translate(floorTransform, floorTransform, [0, 0, 0]);
        - **Translation**: Moves the floor to the origin [0, 0, 0], aligning it with the room's base. No rotation or scaling is applied, suggesting it's already oriented correctly (x-z plane) and sized appropriately.
- **Shader(s) Used and Reasoning**:
    - Vertex Shader: Uses the provided #version 300 es shader with uModelViewMatrix, uProjectionMatrix, uNormalMatrix, uLightPosition, etc.
        - **Reasoning**: Applies perspective projection and lighting (Lambertian model) to simulate realistic illumination.
        - **Accomplishment**: Computes vertex positions in clip space and calculates diffuse lighting based on the normal and light direction, outputting vFinalColor for fragment shading.
    - Fragment Shader: Simple pass-through shader setting fragColor = vFinalColor.
        - **Reasoning**: Keeps rendering lightweight by using pre-computed colors from the vertex shader.
        - **Accomplishment**: Applies the final color (ambient + diffuse) to each pixel.
    - Color set to [0.6, 0.4, 0.2, 1.0] (brown wood) via uMaterialDiffuse.
- **Position and Use of Virtual Camera**:
    - See camera details below for the full description and diagram.

2. Axis

- **Mathematical Characteristics**:
    - A 3D coordinate system representation, typically three lines (x, y, z axes) extending 2000 units from the origin.
    - Defined by line segments or thin quads.
- **Primitives Used to Render**:
    - Lines (likely gl.LINES mode).
- **Transformations Used and Explanations**:
    - No explicit transformation shown; assumed to be at [0, 0, 0] with mat4.identity implicitly applied.
        - **Reasoning**: Serves as a reference frame, so it remains untransformed to align with the world coordinate system.
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders as above.
        - **Reasoning**: Consistent lighting and projection across all objects for uniformity.
        - **Accomplishment**: Renders axes with default or object-specific diffuse color (not explicitly set, so likely uses object.diffuse).
- **Position and Use of Virtual Camera**:
    - See camera details below.

3. RoomFloor

- **Mathematical Characteristics**:
    - A 20x20 unit square plane, likely a quad or mesh.
    - Positioned at y = 0 as the room's base.
- **Primitives Used to Render**:
    - Triangles (two triangles forming a quad).
- **Transformations Used and Explanations**:
    - floorTransform = mat4.create(); mat4.identity(floorTransform); mat4.translate(floorTransform, floorTransform, [0, 0, 0]);
        - **Translation**: Places the floor at the origin, serving as the room's foundation.
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Uniform lighting and projection for room consistency.
        - **Accomplishment**: Renders with [0.6, 0.4, 0.2, 1.0] (brown wood) to match a wooden floor texture.
- **Position and Use of Virtual Camera**:
    - See camera details below.

4. Ceiling

- **Mathematical Characteristics**:
    - A 20x20 unit square plane, parallel to the floor.
    - Positioned at y = 10.
- **Primitives Used to Render**:
    - Triangles (two triangles forming a quad).
- **Transformations Used and Explanations**:
    - ceilingTransform = mat4.create(); mat4.identity(ceilingTransform); mat4.translate(ceilingTransform, ceilingTransform, [0, 10, 0]);
        - **Translation**: Moves the ceiling to y = 10, aligning it with the room's top.
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Consistent rendering approach.
        - **Accomplishment**: Renders with [0.9, 0.9, 0.9, 1.0] (light gray) to simulate a plain ceiling.
- **Position and Use of Virtual Camera**:
    - See camera details below.

5. Left Wall

- **Mathematical Characteristics**:
    - A 20x10 unit rectangle (width x height), rotated 90 degrees around y-axis.
    - Positioned at x = -10.
- **Primitives Used to Render**:
    - Triangles (likely a quad mesh).
- **Transformations Used and Explanations**:

    - leftWallTransform = mat4.create(); mat4.identity(leftWallTransform); mat4.translate(leftWallTransform, leftWallTransform, [-10, 5, 0]); mat4.rotateY(leftWallTransform, leftWallTransform, Math.PI / 2);
        - **Translation**: Moves to x = -10, y = 5 (center height), z = 0.
        - **Rotation**: Math.PI / 2 (90 degrees) around y-axis orients the wall along the z-axis, forming the left boundary.
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Uniform lighting for room walls.
        - **Accomplishment**: Renders with [0.8, 0.8, 0.8, 1.0] (gray) for a neutral wall appearance.
- **Position and Use of Virtual Camera**:
    - See camera details below.

6. Back Wall

- **Mathematical Characteristics**:
    - A 20x10 unit rectangle, unrotated.
    - Positioned at z = -10.
- **Primitives Used to Render**:
    - Triangles (likely a quad mesh).
- **Transformations Used and Explanations**:
    - backWallTransform = mat4.create(); mat4.identity(backWallTransform); mat4.translate(backWallTransform, backWallTransform, [0, 5, -10]);
        - **Translation**: Moves to x = 0, y = 5, z = -10, forming the back boundary.
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Consistent wall rendering.
        - **Accomplishment**: Renders with [0.8, 0.8, 0.8, 1.0] (gray).
- **Position and Use of Virtual Camera**:
    - See camera details below.

7. Broom

- **Mathematical Characteristics**:
    - Complex shape with a brush and handle, approximated by multiple quads or triangles.
    - Dimensions depend on broom.brushHeight and handle length.
- **Primitives Used to Render**:
    - Triangles (likely a mesh of quads).
- **Transformations Used and Explanations**:
    - mat4.identity(broomTransform); mat4.translate(broomTransform, broomTransform, [-9.3, broom.brushHeight / 2, -9.7]); mat4.rotateY(broomTransform, broomTransform, 30 * Math.PI / 180);

mat4.rotateX(broomTransform, broomTransform, -8 * Math.PI / 180);
mat4.rotateZ(broomTransform, broomTransform, 6 * Math.PI / 180);
    - **Translation**: [-9.3, broom.brushHeight / 2, -9.7] places it near the back left corner, lifted by half the brush height.
    - **Rotation Y**: 30 degrees tilts it slightly.
    - **Rotation X**: -8 degrees adjusts the lean.
    - **Rotation Z**: 6 degrees fine-tunes orientation, simulating a leaning broom.
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Uniform lighting for all objects.
        - **Accomplishment**: Renders with [0.0, 0.0, 1.0, 1.0] (blue) to distinguish the broom.
- **Position and Use of Virtual Camera**:
    - See camera details below.

8. Door

- **Mathematical Characteristics**:
    - Rectangular shape with dimensions defined in Door.js (e.g., width, height).
- **Primitives Used to Render**:
    - Triangles (likely a quad mesh).
- **Transformations Used and Explanations**:
    - mat4.identity(doorTransform); mat4.translate(doorTransform, doorTransform, [-6.5, door.height/2, -9.9]);
        - **Translation**: [-6.5, door.height/2, -9.9] positions it on the right wall, centered vertically, slightly offset from the back.
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Consistent room element rendering.
        - **Accomplishment**: Renders with [0.6, 0.4, 0.2, 1.0] (brown wood) to match a wooden door.
- **Position and Use of Virtual Camera**:
    - See camera details below.

9. Table

- **Mathematical Characteristics**:
    - Rectangular or square top with legs, approximated by quads or a mesh.
- **Primitives Used to Render**:
    - Triangles (likely a quad-based mesh).
- **Transformations Used and Explanations**:
    - mat4.identity(tableTransform); mat4.translate(tableTransform, tableTransform, [-1, 0, -9.0]);
        - **Translation**: [-1, 0, -9.0] places it right of the door, on the floor.
- **Shader(s) Used and Reasoning**:

    - Same vertex and fragment shaders.
        - **Reasoning**: Uniform furniture rendering.
        - **Accomplishment**: Renders with [0.6, 0.4, 0.2, 1.0] (brown wood).
- **Position and Use of Virtual Camera**:
    - See camera details below.

10. Microwave

- **Mathematical Characteristics**:
    - Box-like shape, approximated by a cuboid mesh.
- **Primitives Used to Render**:
    - Triangles (likely a cube mesh).
- **Transformations Used and Explanations**:
    - mat4.identity(microwaveTransform); mat4.translate(microwaveTransform, microwaveTransform, [-1.2, tableTopY, -9.0]); where tableTopY = table.height + microwave.height/2.
        - **Translation**: [-1.2, tableTopY, -9.0] places it on the table, centered.
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Consistent appliance rendering.
        - **Accomplishment**: Renders with [0, 0, 0.1, 1.0] (light gray).
- **Position and Use of Virtual Camera**:
    - See camera details below.

11. CouchBase

- **Mathematical Characteristics**:
    - Rectangular base, approximated by a quad or mesh.
- **Primitives Used to Render**:
    - Triangles (likely a quad-based mesh).
- **Transformations Used and Explanations**:
    - mat4.identity(couchBaseTransform); mat4.translate(couchBaseTransform, couchBaseTransform, [-8, couchBase.height / 2, 0]);
        - **Translation**: [-8, couchBase.height / 2, 0] positions it against the left wall, lifted by half its height.
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Uniform furniture rendering.
        - **Accomplishment**: Renders with [0.6, 0.4, 0.2, 1.0] (brown).
- **Position and Use of Virtual Camera**:
    - See camera details below.

12-14. Cushion1, Cushion2, Cushion3

- **Mathematical Characteristics**:

    - Rectangular prisms, defined in CouchCushion.js.
- **Primitives Used to Render**:
    - Triangles (likely a cube mesh).
- **Transformations Used and Explanations**:
    - mat4.identity(cushion1Transform); mat4.translate(cushion1Transform, cushion1Transform, [-8, 1.2, -3.05]);
        - **Translation**: [-8, 1.2, -3.05] for left cushion, etc., positions them on the couch base.
    - Similar for cushion2 ([0]) and cushion3 ([3.05]).
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Consistent cushion rendering.
        - **Accomplishment**: Renders with [0.6, 0.4, 0.2, 1.0] (light brown).
- **Position and Use of Virtual Camera**:
    - See camera details below.

15-17. BackCushion1, BackCushion2, BackCushion3

- **Mathematical Characteristics**:
    - Rectangular prisms, defined in CouchBackCushion.js with varying dimensions.
- **Primitives Used to Render**:
    - Triangles (likely a cube mesh).
- **Transformations Used and Explanations**:
    - mat4.identity(backCushion1Transform); mat4.translate(backCushion1Transform, backCushion1Transform, [-9, 1.2, -1.6]);
        - **Translation**: [-9, 1.2, -1.6] for left, etc., positions them behind the seat cushions.
    - Similar for backCushion2 ([2.3]) and backCushion3 ([5.6]).
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Consistent back cushion rendering.
        - **Accomplishment**: Renders with [0.6, 0.4, 0.2, 1.0] (light brown).
- **Position and Use of Virtual Camera**:
    - See camera details below.

18. ArmrestLeft

- **Mathematical Characteristics**:
    - Complex shape, approximated by a mesh.
- **Primitives Used to Render**:
    - Triangles (likely a quad-based mesh).
- **Transformations Used and Explanations**:
    - mat4.identity(armrestLeftTransform); mat4.translate(armrestLeftTransform, armrestLeftTransform, [-8.3, 0, -5]);
        - **Translation**: [-8.3, 0, -5] positions it at the left end of the couch.

- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Uniform armrest rendering.
        - **Accomplishment**: Renders with [0.6, 0.4, 0.2, 1.0] (brown).
- **Position and Use of Virtual Camera**:
    - See camera details below.

19. ArmrestRight

- **Mathematical Characteristics**:
    - Similar to ArmrestLeft, approximated by a mesh.
- **Primitives Used to Render**:
    - Triangles (likely a quad-based mesh).
- **Transformations Used and Explanations**:
    - mat4.identity(armrestRightTransform); mat4.translate(armrestRightTransform, armrestRightTransform, [-8.3, 0, 5]);
        - **Translation**: [-8.3, 0, 5] positions it at the right end of the couch.
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Consistent armrest rendering.
        - **Accomplishment**: Renders with [0.6, 0.4, 0.2, 1.0] (brown).
- **Position and Use of Virtual Camera**:
    - See camera details below.

20. TrashCan

- **Mathematical Characteristics**:
    - Conical frustum (open-top cylinder) with base radius 0.5, top radius 0.75, height 2.
- **Primitives Used to Render**:
    - Triangles (mesh of triangular faces approximating the cone).
- **Transformations Used and Explanations**:
    - mat4.identity(trashCanTransform); mat4.translate(trashCanTransform, trashCanTransform, [-5, trashCan.height / 2, -5]);
        - **Translation**: [-5, 1, -5] (height/2 = 1) places it near the back left corner.
- **Shader(s) Used and Reasoning**:
    - Same vertex and fragment shaders.
        - **Reasoning**: Uniform object rendering.
        - **Accomplishment**: Renders with [0.3, 0.3, 0.3, 1.0] (dark gray) for a metallic look.
- **Position and Use of Virtual Camera**:
    - See camera details below.

Position and Use of Virtual Camera

- **Initial Position and Orientation**:
    - Initialized in configure with camera.goHome([0, 5, 8]); in orbiting mode.
    - **Position**: [0, 5, 8] (x, y, z) places the camera 8 units along the z-axis, 5 units above the floor, centered on x.
    - **Orientation**: Orbiting mode suggests it rotates around a target (likely [0, 0, 0]), with elevation and azimuth controllable via initControls.
    - **Field of View (FOV)**: fov = 45, providing a moderate wide-angle view.
    - **Projection Mode**: Starts as PERSPECTIVE_PROJECTION, switchable to ORTHOGRAPHIC_PROJECTION.
- **Use**:
    - Allows user interaction via Controls to adjust position, rotation (elevation/azimuth), and dolly (zoom).
    - goHome() resets to [0, 5, 8], ensuring a default view of the room.